

INF5130 - Algorithmique

Série d'exercices - 1

1 Exercice 1

On considère le problème consistant à additionner deux nombres naturels exprimés en **base deux**, dont les nombres sont représentés dans des tableaux de n éléments. Ces tableaux sont respectivement appelés A et B . La somme des deux nombres doit être calculée et placée dans un tableau nommé C et contenant $n + 1$ éléments.

- a) Écrivez une fonction qui prend en entrée les tableaux A et B et renvoie le tableau C .
- b) Donnez la complexité temporelle de votre algorithme.

2 Exercice 2

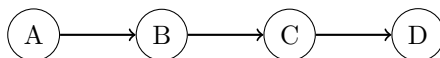
Soient A et B deux tableaux ayant la même signification que dans l'exercice précédent. On veut maintenant multiplier les nombres représentés par A et B au lieu de les additionner. Le tableau C contiendra la représentation du produit de A et B en base deux.

- a) Quel doit être le nombre d'éléments du tableau C ?
- b) Écrivez une fonction qui prend en entrée A et B et renvoie C .
- c) Donnez la complexité temporelle de votre algorithme.

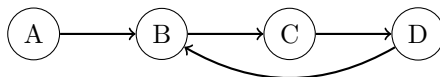
3 Exercice 3

Étant donnée une liste chaînée L , cette dernière est cyclique si et seulement s'il existe un maillon pour lequel il existe un chemin permettant de retourner à ce dernier.

Par exemple, la liste chaînée suivante ne possède pas de cycle :



En revanche, cette liste chaînée en possède un :



Écrivez un algorithme qui renvoie **vrai** si la liste chaînée T contient un cycle, sinon l'algorithme renvoie **faux**. Contrainte : votre algorithme doit être dans le pire des cas en $\mathcal{O}(n)$ pour la complexité temporelle et $\mathcal{O}(1)$ en complexité spatiale.

Pour cette exercice, vous avez accès seulement à ces fonctions qui sont exécutées en temps constant :

- OBTENIR_TETE(L) : renvoie l'emplacement du premier maillon de la liste chaînée L .
- OBTENIR_SUIVANT(m) : renvoie le maillon suivant m . S'il le maillon m ne contient pas de suivant, alors la fonction renvoie la valeur *None*.

4 Exercice 4

Soit l'algorithme de tri nommé TRI COMPTAGE est décrivable par l'algorithme suivante :

```

1  def tri_comptage(A, k):
2      C = [0] * (k + 1)           # c1
3      B = [0] * (len(A))          # c2
4      j = 0                       # c3
5      while j < len(A):           # c4
6          C[A[j]] = C[A[j]] + 1   # c5
7          j = j + 1               # c6
8      j = 1                       # c7
9      while j <= k:               # c8
10         C[A[j]] = C[j] + C[j - 1] # c9
11         j = j + 1               # c10
12     j = len(A) - 1              # c11
13     while j >= 0:               # c12
14         B[C[A[j]]] = A[j]       # c13
15         C[A[j]] = C[A[j]] - 1   # c14
16         j = j - 1               # c15
17     return B                    # c16
  
```

Ce dernier a la particularité de donner un tableau trié sans effectuer de comparaisons. Une des contraintes de cet algorithme de tri est que le tableau qui doit être trié ne contient seulement que des nombres entiers telles que $0 \leq n \leq k$.

- a) Donnez sa complexité temporelle en fonction de $c_1, c_2, c_3, c_4, c_5, c_6, c_7, c_8, c_9, c_{10}, c_{11}, c_{12}, c_{13}, c_{14}, c_{15}$ et c_{16} .
- b) Donnez sa complexité spatiale.

5 Exercice 5

Considérez la procédure suivante :

```

1  def werk(M):
2      somme = 0
3      i = 0
4      while i < len(M):
5          stack = 0
6          j = i + 1
7          while j < len(M):
8              stack = stack + M[i][j]
9              M[i][j] = 0
10             j = j + 1
11             rest = stack % (i + 1)
12             stack = stack // (i + 1)
13             p = 0
14             while p <= i:
15                 M[i][p] = M[i][p] + stack
16                 if p == i:
17                     M[i][p] = M[i][p] + rest
18                 p = p + 1
19             i = i + 1

```

- a) Expliquez ce que fait cet algorithme.
- b) Identifiez une instruction barométrique et expliquez pourquoi il s'agit d'une opération barométrique.
- c) Donnez la complexité de cet algorithme en se basant sur l'opération barométrique trouvée à la question précédente.

6 Exercice 6

Soit le problème suivant pour lequel on souhaite obtenir la somme de tous les éléments présents sur la diagonale principale et la diagonale secondaire d'une matrice M carrée.

- a) Écrivez un algorithme permettant de résoudre ce problème.
- b) Selon l'algorithme écrit à la question précédente, identifiez une opération barométrique et expliquez ce choix.

- c) Donnez la complexité de votre algorithme en se basant sur l'opération barométrique choisie à la question précédente.